

parallelization summary for lecture #7

type	split	benefits	how it works	key trade-off
DP	training data	increase throughput	each GPU holds a full model copy, process different data samples in parallel, gradients are averaged (all-reduce)	no memory saving, large sync/grad cost
TP	Tensors within layers (matrix rows/columns)	Enable very large layers that exceed one GPU's memory	Each GPU stores and computes part of a single layer's tensor, e.g. split GEMMs or Attention heads	heavy intra-layer communication (All-Reduce/All-gather)
PP	layers across GPUs	Reduce memory per GPU	Different GPUs hold different sequential layers; micro-batch flow through them like an assembly line	Pipeline bubbles (waitly time) and load imbalance to reduce efficiency
SP	tokens in sequence dimension	optimize memory and communications for long sequence	Split the sequence dim across GPUs, so each process a subset of tokens [used in attention compute]	increase spec frequency, need All-gather for context mixing
EP	Experts/MLPs in MoE layers	Scale model capacity without increasing per-token compute	Each GPU hosts different Experts; only a subset expert is activated per input token	Require dynamic routing and load balancing, communication spikes

takeaways:

- 1): scaling beyond a certain point requires multi-GPU / multi-node parallelism
 - 2): no single solution to the parallelism (probably want all 3 approaches)
 - 3): Simple, interpretable rules of thumb for combining different forms of parallelism
 - 3D parallelism := DP + TP + PP
 - 4D parallelism = DP + TP + PP + EP (for MoE models)
- SP is often used as an optimization inside TP/PP to reduce Activation memory of Attention